Ex.no :10 **Pneumonia Detection from Medical Images using Machine Learning**
Date : *(MedVision-ML Clinical Computer Vision & Machine Learning System)*

Problem Statement:

Pneumonia is a serious respiratory infection that can produce abnormal patterns in medical images. Manual interpretation of radiological images requires trained medical professionals and can be time-consuming, especially when a large number of cases must be screened. A computer-aided system can support preliminary analysis by applying image preprocessing, quantitative feature extraction and machine-learning classification.

The MedVision-ML project provides an end-to-end medical imaging demonstrator in which an input image is loaded, preprocessed, converted into numerical radiological descriptors, classified using a machine-learning algorithm, and evaluated using standard performance measures. The interface is organized as five stages: Load Input Image, Preprocessor, Feature Extraction, Model Training, and Performance & Diagnosis.

Objectives:

1. To load and organize normal and pneumonia medical-image samples for classification.
2. To preprocess the input image by grayscale conversion, resizing, denoising, contrast enhancement and intensity normalization.
3. To extract radiological features using GLCM texture measures, statistical intensity descriptors and Sobel/Laplacian edge information.
4. To train a machine-learning classifier and provide a selectable model-training interface.
5. To evaluate the trained classifier using Accuracy, Sensitivity (Recall), Specificity, Precision, F1-Score, ROC-AUC and a Confusion Matrix.
6. To provide a web-based diagnostic demonstrator that displays the predicted class and confidence for the active scan.

Software / Hardware Required:

- Operating System: Windows / Linux / macOS
- IDE: Visual Studio Code / Antigravity IDE
- Language: Python 3.x
- Web Application: HTML5, CSS3, JavaScript with a Python backend
- Machine Learning: scikit-learn
- Image Processing: OpenCV (cv2), NumPy and related image-processing utilities
- Dataset: Normal and Pneumonia medical-image samples
- Model Support: SVM, Random Forest, K-Nearest Neighbors and Logistic Regression

Project Directory Structure:

The MedVision-ML project is organized as a Python web application with model files, static front-end resources, benchmark samples, templates and supporting training scripts, as shown in the uploaded development-environment screenshot.

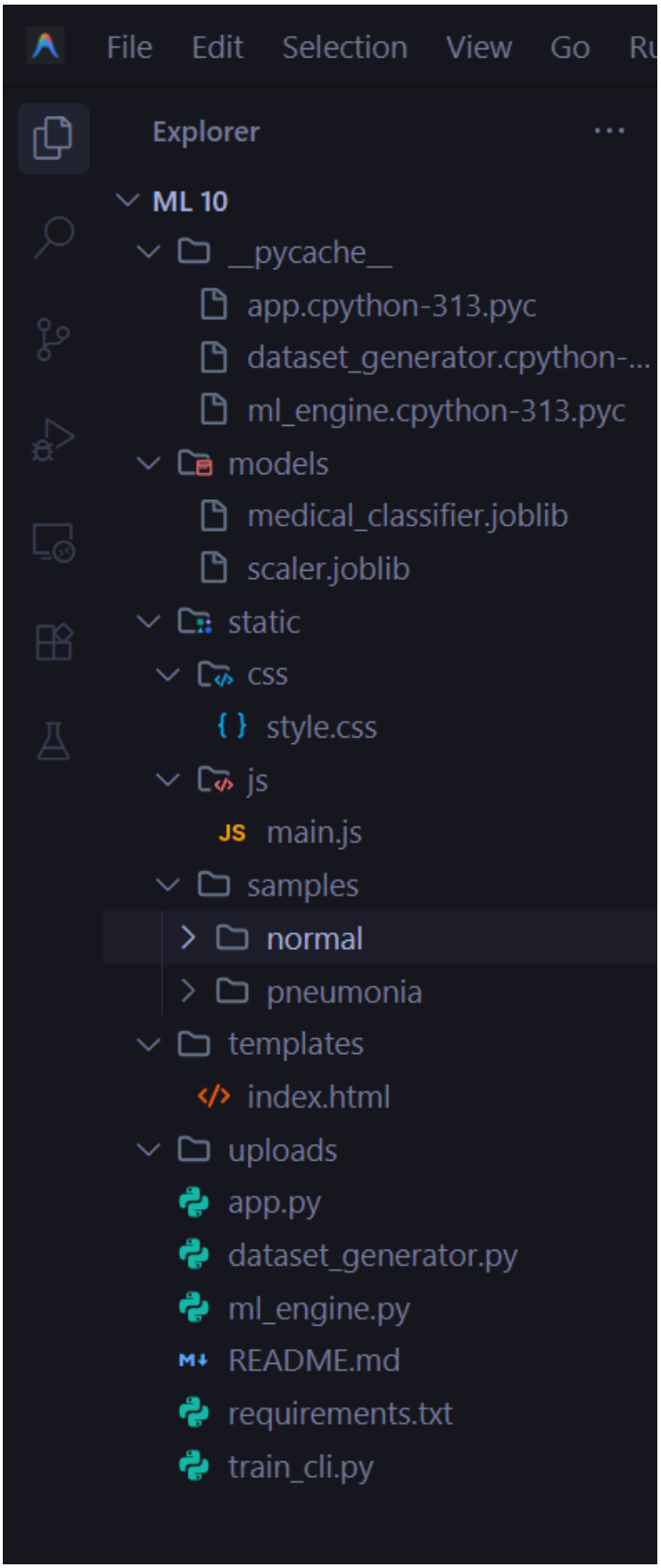


Fig 1: Project directory structure of the MedVision-ML project in the development environment

Procedure:

1. Prepare labelled Normal and Pneumonia image samples and make them available to the application.
2. Launch the Python web application and open the MedVision-ML interface in a browser.
3. Load an input image or select one of the built-in benchmark cases (Step 1).
4. Preprocess the image by converting it to grayscale, resizing it to 128×128 pixels, applying a 3×3 median filter, enhancing contrast using histogram equalization, and normalizing intensity values to the [0, 1] range (Step 2).
5. Extract a radiological feature vector using GLCM texture descriptors, first-order statistical descriptors, Sobel edge energy and Laplacian variance (Step 3).
6. Select a machine-learning algorithm and configure the test split in the Model Training stage.
7. Train the classifier on the available feature vectors and obtain the trained-model status and accuracy (Step 4).
8. Evaluate the trained classifier using the held-out test set and display the confusion matrix, ROC curve and diagnostic metrics (Step 5).
9. Run the clinical diagnosis for the active scan and display the predicted class and confidence level.
10. Interpret the displayed result only as a project demonstration / decision-support output and not as a standalone clinical diagnosis.

Implementation Code:

Representative implementation of the key image-processing module based on the pipeline shown in the application screenshots.

1. Image Preprocessing (ml_engine.py)

```
import cv2
import numpy as np


def preprocess_image(path, target_size=(128, 128)):
img = cv2.imread(path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img = cv2.resize(img, target_size)
img = cv2.medianBlur(img, 3)
img = cv2.equalizeHist(img)
img = img.astype(np.float32) / 255.0
return img
```

2. Feature Extraction (ml_engine.py)

```
import cv2
import numpy as np
from skimage.feature import graycomatrix, graycoprops

def extract_features(img):
u8 = (img * 255).astype(np.uint8)

q = (u8 // 16).astype(np.uint8)
glcm = graycomatrix(q, [1], [0], levels=16,
symmetric=True, normed=True)

texture = [
graycoprops(glcm, "contrast")[0,0],
graycoprops(glcm, "dissimilarity")[0,0],
graycoprops(glcm, "homogeneity")[0,0],
graycoprops(glcm, "energy")[0,0],
graycoprops(glcm, "correlation")[0,0]
 ]

stats = [
float(np.mean(img)),
float(np.std(img)),
float(np.mean((img-img.mean())**3)),
float(np.mean((img-img.mean())**4)),
float(-np.sum(img * np.log(img + 1e-8)))
 ]

gx = cv2.Sobel(u8, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(u8, cv2.CV_64F, 0, 1, ksize=3)
edge = np.sqrt(gx**2 + gy**2)

sobel = [float(edge.mean()), float(edge.std())]
lap = cv2.Laplacian(u8, cv2.CV_64F)
lap_var = float(lap.var())

return np.asarray(texture + stats + sobel + [lap_var],
dtype=np.float32)
```

3. Model Training (train_cli.py)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
import joblib
import numpy as np

X = np.load("processed/features.npy")
y = np.load("processed/labels.npy")

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.30, stratify=y, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

joblib.dump(model, "models/medical_classifier.joblib")
joblib.dump(scaler, "models/scaler.joblib")
print("Model trained successfully")
```

4. Prediction / Diagnosis

```
import joblib
import numpy as np

model = joblib.load("models/medical_classifier.joblib")
scaler = joblib.load("models/scaler.joblib")
CLASSES = ["Normal", "Pneumonia"]

def predict_scan(image_path):
img = preprocess_image(image_path)
features = extract_features(img).reshape(1, -1)
features = scaler.transform(features)

prediction = model.predict(features)[0]
if hasattr(model, "predict_proba"):
proba = model.predict_proba(features)[0]
confidence = float(np.max(proba)) * 100
```

```
else:
    confidence = 0.0

return {
 "prediction": CLASSES[int(prediction)],
 "confidence": round(confidence, 2)
  }
```

5. Performance Evaluation

```
from sklearn.metrics import (
accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, roc_auc_score
 )

pred = model.predict(X_test)

accuracy = accuracy_score(y_test, pred)
precision = precision_score(y_test, pred)
recall = recall_score(y_test, pred)
f1 = f1_score(y_test, pred)
cm = confusion_matrix(y_test, pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
print("Confusion Matrix:")
print(cm)
```

Model Training Configuration Shown in the Interface:

- Final demonstrated classifier: K-Nearest Neighbors (k = 5).
- Test split: 30% (70% training / 30% testing).
- Total samples shown after training: 30.
- Training samples: 21; test samples: 9.
- The interface also presents SVM, Random Forest and Logistic Regression as selectable algorithms.

Output (Step-by-Step Screenshots):

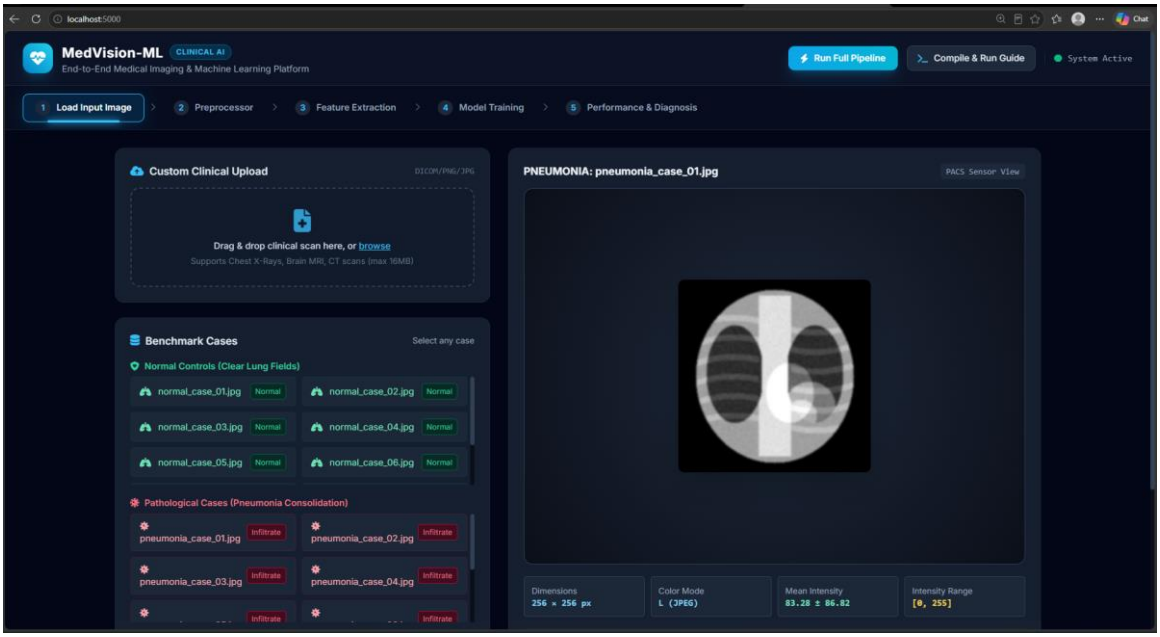


Fig 2: Step 1 — Load Input Image: interface for uploading a clinical scan or selecting a benchmark case



Fig 3: Step 1 — Input image preview, benchmark cases and radiodensity histogram

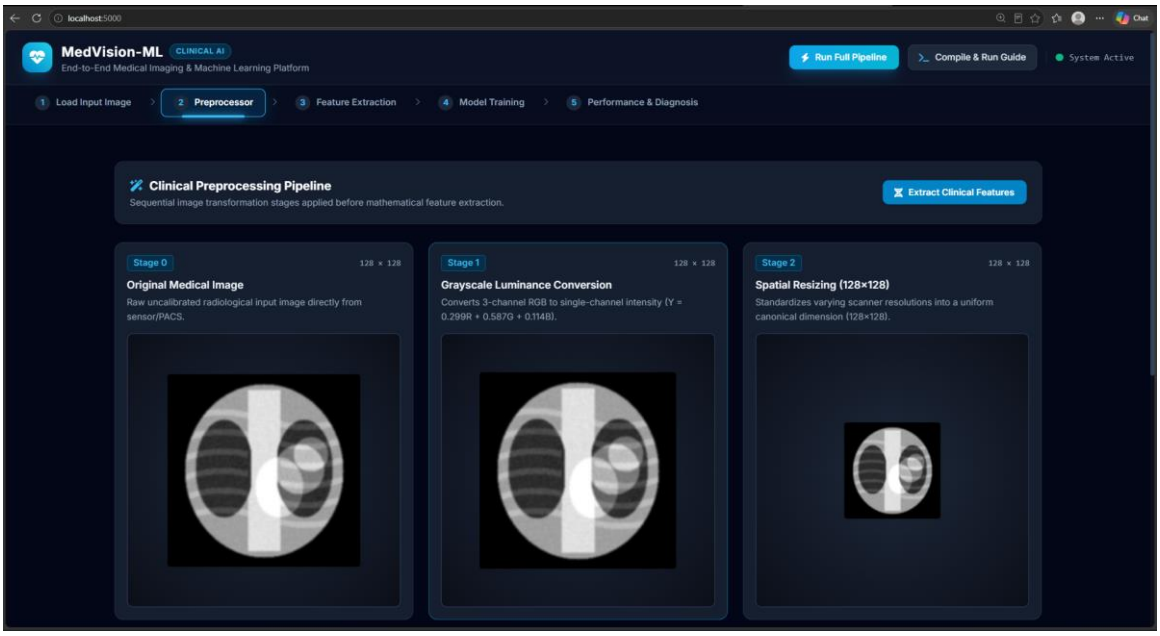


Fig 4: Step 2 — Clinical preprocessing pipeline: original image, grayscale luminance conversion and 128×128 spatial resizing

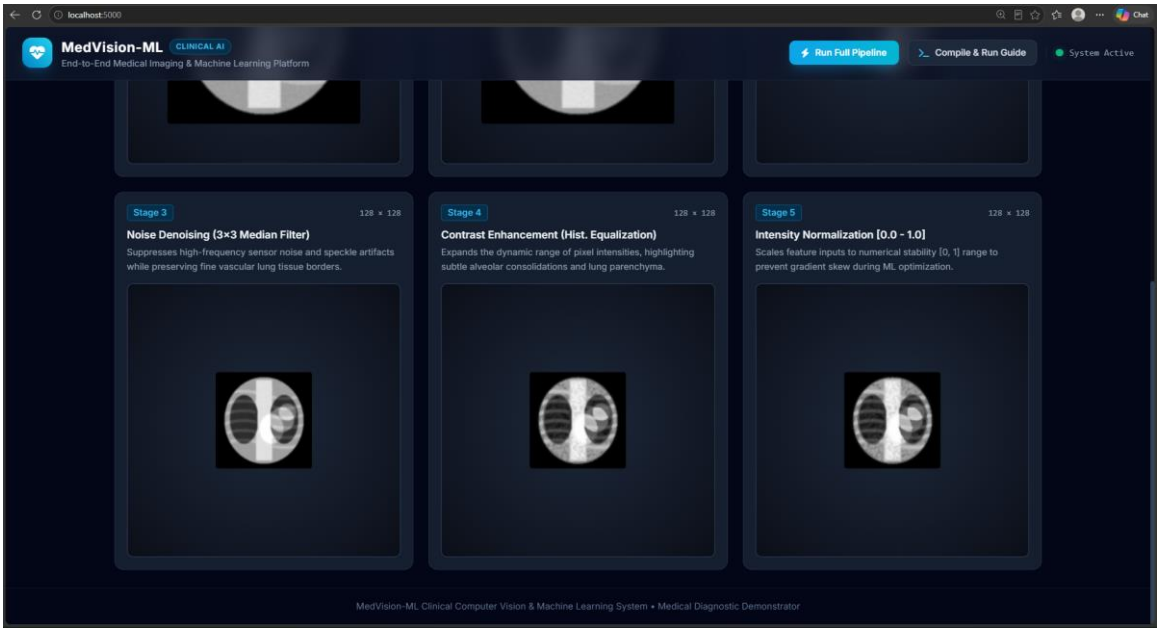


Fig 5: Step 2 — Noise denoising, contrast enhancement and intensity normalization stages

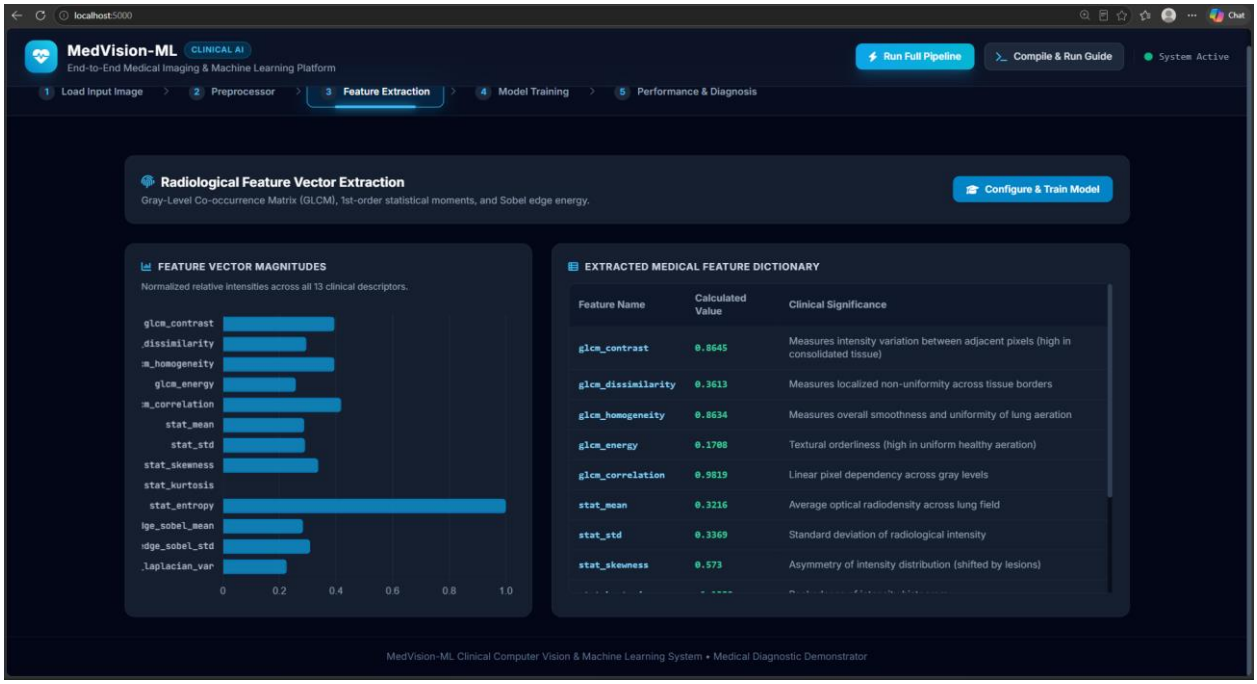


Fig 6: Step 3 — Radiological feature vector extraction: GLCM texture, statistical and edge-based descriptors

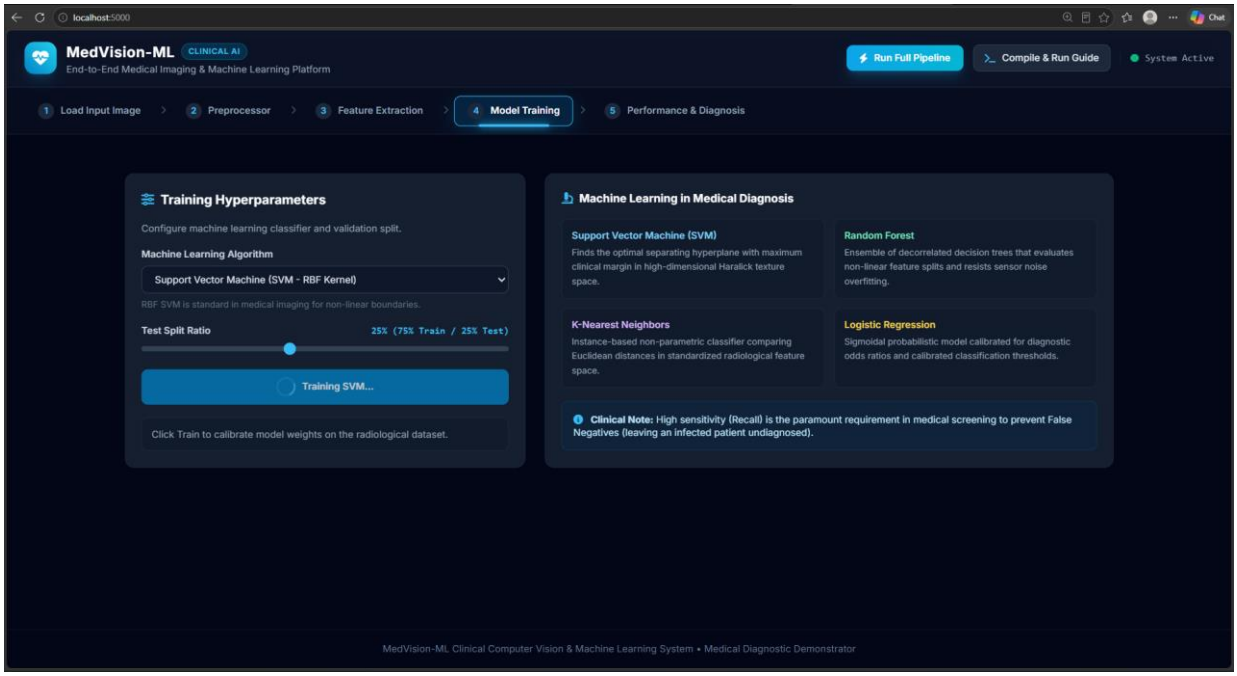


Fig 7: Step 4 — Model Training: classifier selection and test-split configuration

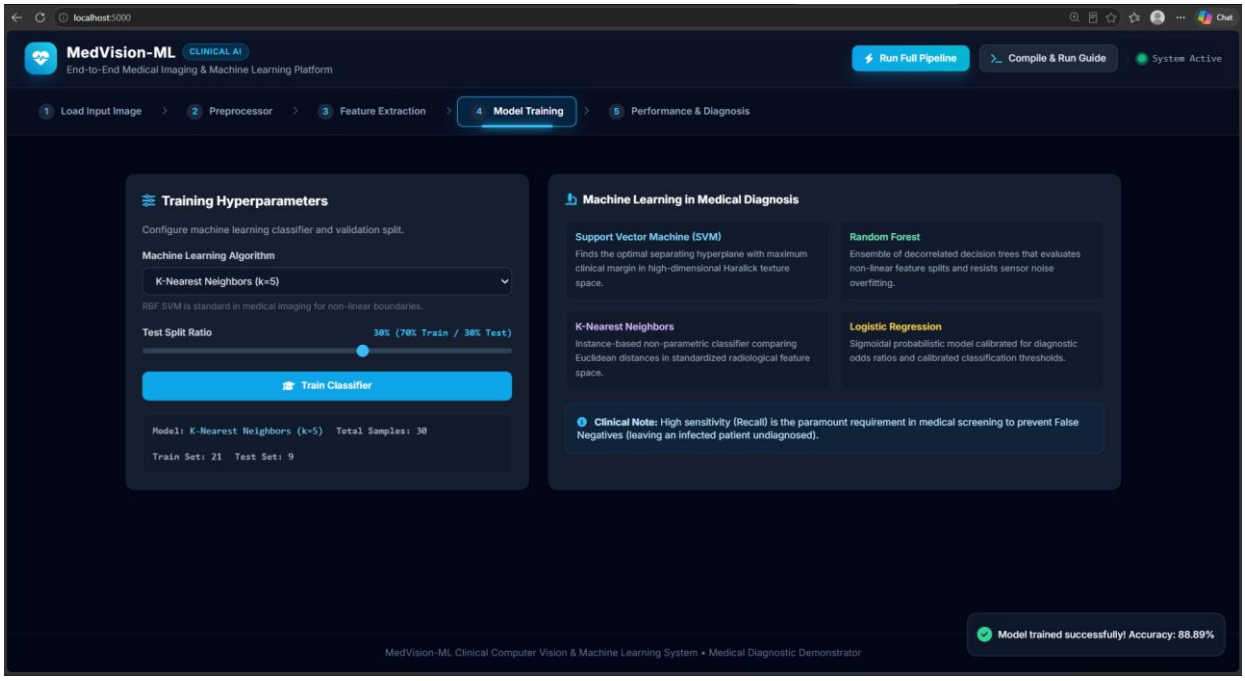


Fig 8: Step 4 — Model Training completed successfully: K-Nearest Neighbors (k=5), 70% training and 30% testing

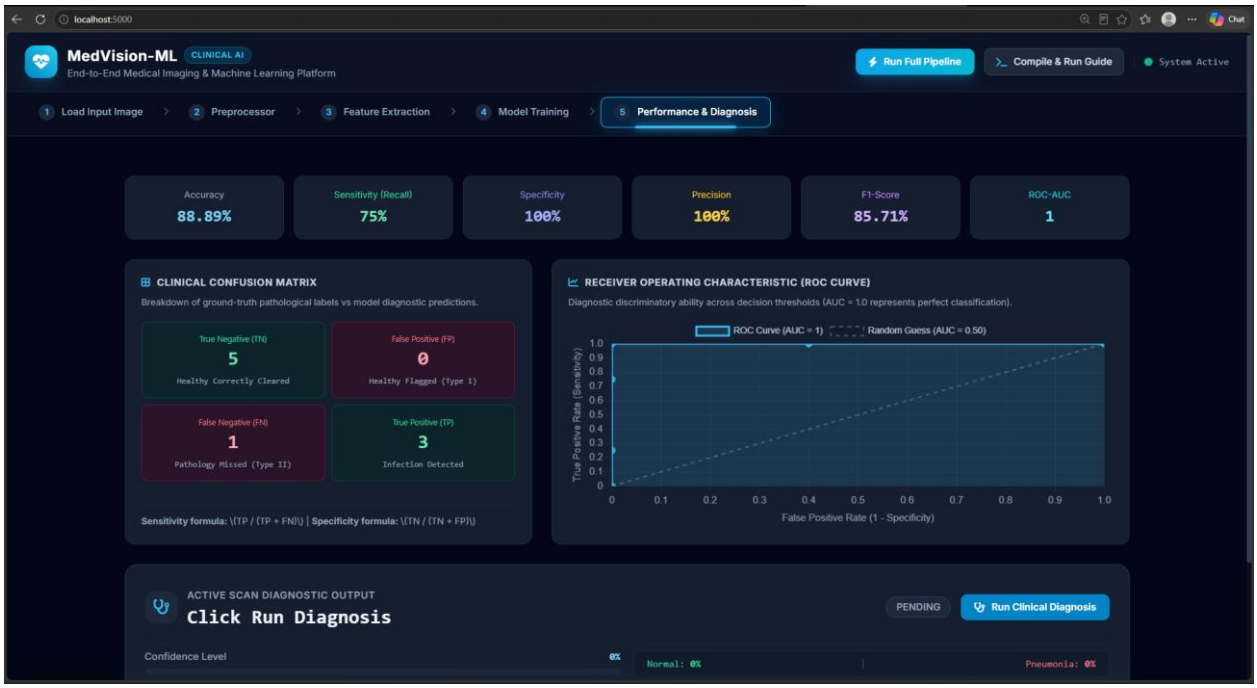


Fig 9: Step 5 — Performance & Diagnosis: accuracy, sensitivity, specificity, precision, F1-score, ROC-AUC and confusion matrix



Fig 10: Active Scan Diagnostic Output — Pneumonia detected with the displayed confidence and clinical-action recommendation



Fig 11: Final diagnostic result screen — Pneumonia detected, with radiological finding and clinical-action recommendation

Result:

A web-based MedVision-ML medical imaging demonstrator was successfully designed with an end-to-end pipeline covering image loading, preprocessing, radiological feature extraction, machine-learning model training, performance evaluation and clinical diagnostic output. The preprocessing pipeline standardizes the image to 128×128 pixels and applies grayscale conversion, median denoising, histogram equalization and intensity normalization.

The final interface successfully displays a Pneumonia detected result for the active scan together with a confidence indicator, radiological finding and clinical-action recommendation. The application is intended as an educational and demonstration system; the displayed prediction should not be treated as a substitute for professional medical diagnosis.